

AIWA Yellow Paper

Causal Coordination and Local Value Accrual for Partition-Tolerant Networks

Version 2.0 — formal specification, reference implementation

Abstract

AIWA is a value-accrual and causal-coordination primitive for networks under arbitrary delay, intermittent connectivity, and unbounded partition. It separates two properties conventionally coupled in distributed ledgers:

Coordination. A deterministic causal layer over authenticated events, rooted in a common genesis. Domains operate while disconnected; reconciliation is deferred, not required.

Accrual. Local, unconditional creation of value, gated at genesis by an external commitment (§8) and thereafter driven by real sequential computation (§6) — never by a shared clock.

Progression → claimable value Conservation → ownership Mirror → verifiable history

No component requires a globally synchronized state.

1 System model

A domain is an operational environment holding one or more identities and local state. An identity is a keypair. Communication between domains is continuous, intermittent, delayed, asymmetric, or absent, for arbitrary duration.

Assumed: collision-resistant hashing, EUF-CMA-secure signatures, deterministic serialization. **Not assumed:** a synchronized wall clock, continuous access to any external chain, a global consensus quorum, a globally replicated state.

2 Identity

$$\text{domain}(i) = \text{SHA-256}(\text{pk}_i)$$

Full 256-bit digest, hex-encoded, untruncated. The private key authorizes state transitions; no other binding — device, IP, location — is part of identity.

3 Event DAG

State transitions are signed events referencing causal parents, forming a DAG. A participant need not hold every event, only what is relevant to its own state and observed relationships. $G_1 \cup G_2$, for two independently-grown DAGs, is a commutative union.

4 Mirror

$M_D \in \{\text{empty}, \text{full}\}$: a domain D 's own signed, per-epoch commitment to what it has observed of another domain, never a replica of the observed state.

Reception monotonicity. For domain D observing X : $\text{seen}_D(X, e_{i+1}) \geq \text{seen}_D(X, e_i)$ for successive real commitments e_i . A claim of having seen less than previously committed is rejected.

Mirror does not establish that two domains are distinct real-world actors, nor rule out a coalition fabricating consistent history together at real cost. It is evidence about the structure of observed history — not an identity oracle.

5 Progression

$$\text{epoch}_D(n+1) = \text{epoch}_D(n) + 1$$

valid only if causally chained to D 's own last accepted transition and carrying a real sequential proof (§6). Progression is local: epoch_A and epoch_B are never directly comparable.

6 Sequential proof

$$h_0 = \text{SHA-256}(\text{seed}), \quad h_i = \text{SHA-256}(h_{i-1})$$

seeded from $(\text{domain}, \text{output}_{n-1})$. Symmetric — verification cost equals production cost — unlike an asymmetric VDF (Wesolowski, Pietrzak); what is unconditional is that production cannot be parallelized or shortcut by any amount of hardware.

Locality. Computation of h_i occurs on exactly one real device at a time — never distributed, never assisted by other domains. Mirror and live sync (§4, §13) carry only already-computed results. Identity is the keypair (§13.1); the computing device may change without discontinuity in epoch_D .

Rate, not calendar time. A fixed-length chain ($\text{VDF_ITERATIONS} = 12000$ in this deployment) measures $\sim 256\text{ms}$ of real computation — a floor, not a symmetric clock: real hardware differs across domains, so epoch_D is a true measure of D 's own sequential work, correlated with but not equal to elapsed real time for any other domain. Comparing raw epoch counts across domains, or adjusting one domain's earned value against another's, would either discount genuine work under weaker hardware or credit idle time for merely reconnecting (§13).

7 Accrual

$$R(S, t, A) = \frac{S \cdot t^\alpha}{[\beta \ln A + \ln(1 + C/A^\beta)]^\gamma}$$

Symbol	Meaning
S	committed capital (§8, cumulative)
t	epochs since D 's own last economic action (burn or claim); resets on each
A	D 's own total progression age; never resets
α, β, γ, C	deployment parameters

Invariant. t and A are derived exclusively from D 's own verified progression state at query time — never accepted from an event payload. A caller-supplied reference epoch would permit unbounded t .

t resetting on claim encodes patience *since last action*, not since genesis; A , unreset, is a maturity denominator independent of claiming frequency.

Computable form: $\beta \ln A + \ln(1 + C/A^\beta) \equiv \ln(A^\beta + C)$, represented in the left form to bound overflow for large A .

8 Genesis Commitment

$$b_D = \sum_{\text{valid burns}} \text{lamports}_i$$

Activation requires an irreversible SOL burn to Solana’s incinerator address, verified from a finalized transaction record. Cumulative across every valid burn, each independently checked against the deployment’s own churn cost curve at its own slot — a later commitment is never grandfathered at an earlier, lower requirement.

R is linear in S : absent a genesis cost, splitting capital across identities would not reduce total accrual. A per-identity activation cost makes churn strictly costlier, not free; whether a given $(\alpha, \beta, \gamma, C, \text{cost curve})$ tuple makes churn net-unprofitable — not merely costly — is computed, not asserted (`churn-analysis.js`).

External dependency. Broadcasting a burn, or any further Solana-side action, requires reaching a centralized, Earth-hosted RPC endpoint over real internet — the one exception to this document’s own no-shared-infrastructure principle. Once activated, epoch_D requires no further contact with Solana or Earth.

9 Conservation

A claimed unit is discrete and owned. Transfer requires a signature proving control over the source; split divides one claim into two whose amounts sum exactly to the original, by construction (integer subtraction), never a separately-checked invariant. Neither operation depends on continued access to progression or the activation chain.

10 Denomination

1 AIWA = 10^{18} base units, always integer. Reward’s fractional exponents require floating-point computation internally; the one real boundary where a value becomes part of a balance converts to an exact integer faithful to the float’s own bits.

11 Partition and reconciliation

Each domain continues independently through partition; none decrements state for another’s unreachability, none awaits permission. Reconciliation is signature, ancestry, and Mirror-observation verification over newly available evidence — never a question of clock authority.

Channel versus data. A transport session (this reference implementation’s own manually-bootstrapped WebRTC, or any real link) is inherently temporary — it exists only while both ends are simultaneously reachable, precisely like a real interplanetary link during a real blackout. Once real data has crossed it, that data is verified and stored durably by each side independently; losing the session never loses what already crossed. Re-establishing a channel after a gap is a real, ordinary event, not a failure — this reference implementation’s own manual handshake is a placeholder for whatever real transport (DTN, dedicated hardware) a real deployment would use; the reconciliation logic it feeds (§4, §13) is agnostic to which.

11.1 Positioning

Published interplanetary-cryptocurrency proposals generally extend one Earth-anchored consensus chain across the latency gap — DTN transport, timelocks widened to light-time, federated or merge-mined settlement — leaving consensus and issuance untouched. This transfers already-created value under latency; it leaves creation itself dominated by whichever side has more compute (mining from Mars against Earth’s hashpower is acknowledged, in that literature, as structurally unprofitable).

AIWA removes value creation from any consensus chain: epoch_D requires no awareness of one. Reconciliation (§4, §13) is additive and informational only. This closes the specific asymmetry above; it does not address real byte transport under latency (reused, not reinvented — `p2p-connection.js`) nor an exchange rate between economies that grew apart.

12 Explicit non-claims

Not solved: the human-identity oracle, physical-location verification, absolute global time, Byzantine agreement without assumptions, detection of every coalition of identities under one real actor. A coalition can produce internally consistent history at real cost. The claim is narrower: fabricated identities cannot fabricate authenticated history *for free*.

13 Causal Tick

A domain's own epoch_D (§5) is unconditional and requires zero external observers. Causal Tick is a complementary, externally-corroborated position — useful for reconciliation and for flagging drift between self-report and observation, never a substitute.

$$\hat{\theta}_X = \text{median}_w(\{(\text{obs}_i(X), w_i)\}), \quad w_i = b_i \text{ (§8)}$$

One estimate per real observer — their own most-recent, highest-resolving observation of X — never one per historical commitment referencing the same fact; an earlier version double-counted replayed observations, closed here. $[\min_i \text{obs}_i, \max_i \text{obs}_i]$ is reported alongside $\hat{\theta}_X$, not discarded. Robust while adversarial weight stays below $\sum w_i/2$ — proof-of-stake's own security shape, applied to causal position.

$\hat{\theta}_X = \perp$ with zero observers — honest absence, not inconsistency.

Domain invariance. For identical evidence, $\hat{\theta}_X$ is identical regardless of which domain computes it or via which path evidence arrived — a consequence of event ids already being content-addressed (§3): identical causal content produces an identical id independent of transport.

Rejected alternative. A structural diversity measure (distinct branches of new causal content) was considered in place of $w_i = b_i$ and rejected: indistinguishable from a well-resourced actor fabricating many distinct, never-replayed branches — verified concretely (three genuine sources and three sybil-fabricated sources produce identical Shannon entropy). Real committed capital remains the only weight source that closes this.

Never a correction. $\hat{\theta}_X$ is reported, never applied to D 's own earned value. Reducing a slower domain's real value toward a faster peer's would punish genuine work under weaker hardware (§6); inflating a reconnecting domain's value toward a peer's would credit elapsed idle time and is directly exploitable by deliberate isolate-then-reconnect cycles. Neither is implemented.

13.1 Hardware roots (optional)

The evidence interface functions on software primitives alone. A domain may optionally strengthen independence assurance with physically-provisioned hardware roots; hardware never computes $\hat{\theta}_X$, never scales w_i , never becomes required input. Zero versus several hardware attestations yield an identical $\hat{\theta}_X$ from identical Mirror evidence — only a separate, informational count differs.

Attestation. A two-hop signature chain: an origin domain signs issuance of a hardware root; the root's own key signs its binding to a target domain. Both independently verified. ≥ 2 distinct, independently-issued roots required — one root moves trust to a single unit, never establishes a minimum.

Stated limit. No cryptographic protocol verifies from software alone that a root's key lives on genuinely non-clonable hardware. What is real: a traceable signature chain from a known origin, reducing trust surface from *any of many sybils* to *the origin's own issuance process, or one physical unit* — a real reduction, not an absolute guarantee.

Identity is the keypair, never a device: loss of hardware is real but never identity loss, restored unchanged from a saved key or passphrase (§8) on any reachable device. This closes device loss, not the case this document is otherwise built around — a domain with no reachable device or channel at all, for a real, extended period. No device, old or new, has anything to reconnect to in that case, regardless of key; hardware roots strengthen assurance for a domain that *is* reachable, and do not substitute for reachability during a real partition.

Reference implementation

Concept	File
Identity	<code>public/core/domain-id.js</code>
Event DAG	<code>public/core/event-dag.js</code>
Sequential proof (§6)	<code>public/core/vdf.js</code> , <code>public/core/wesolowski-vdf.js</code> , <code>public/core/bigint-math.js</code>
Progression (§5)	<code>public/core/progression.js</code>
Accrual formula (§7)	<code>public/core/reward.js</code>
Accrual position, t/A	<code>public/core/accrual.js</code>
Genesis Commitment (§8)	<code>public/core/identity-cost.js</code> , <code>public/core/solana-wallet.js</code>
Churn profitability check	<code>public/core/churn-analysis.js</code> — parameter-specific, not a general guarantee
Cross-runtime interoperability	<code>interop/rust-vdf/</code> — independent Rust, byte-identical to JS output
Conservation (§9)	<code>public/core/conservation.js</code>
Denomination (§10)	<code>public/core/units.js</code>
Mirror (§4)	<code>public/core/mirror.js</code>
Causal Tick (§13)	<code>public/core/causal-tick.js</code> , <code>public/core/weighted-median.js</code>
Hardware roots (§13.1)	<code>public/core/hardware-attestation.js</code>
Live peer-to-peer sync	<code>public/core/p2p-signaling.js</code> , <code>public/app/p2p-connection.js</code>
Coherent composition	<code>public/core/wallet.js</code>

229 tests; every case but the cross-runtime check runs unconditionally, which skips (never fails) without a Rust toolchain. Security-relevant cases are named as such in their own files.